

Spring Project: Festival Pulse

This exercise brings together your Spring Boot learning in a small one-day project.

Work in groups of 2 or 3.

The **minimum requirement** is a working Spring Boot REST API and a simple browser-based UI. Database persistence and Kafka are optional extensions.

Scenario

You are building **Festival Pulse**, a simple dashboard for a small music and food festival. Festival stewards can report how busy different areas are. If an area becomes too crowded, the application creates an alert for the organisers. The project should be small, useful, and easy to demo.

Core Domain

The application has three concepts:

- **Festival Area** — a place at the festival
- **Crowd Report** — a report saying how busy an area is
- **Crowd Alert** — an alert created when an area is too busy

Minimum Requirements

Festival Areas

The system must allow users to:

- Create a festival area
- View all festival areas

An area should include enough information to identify it, such as:

- Name
- Description or location
- Area type

Example areas: Main Stage, Food Village, Craft Beer Bar, First Aid Point

Crowd Reports

The system must allow users to:

- Submit a crowd report for an existing area
- View recent crowd reports

A crowd report should include:

- Festival area

- Crowd level
- Short note
- Time submitted

Crowd level should include: LOW, MEDIUM, FULL

Crowd Alerts

The system must create an alert when a 'FULL' crowd report is submitted.

The system must allow users to:

- View active alerts
- Resolve an alert

An alert should include: Festival area, Alert message, Status, Time created

REST API Requirements

Create REST APIs to support the minimum workflows:

- Create and list festival areas
- Submit and list crowd reports
- List and resolve crowd alerts
- Retrieve a simple dashboard summary

You should decide endpoint paths, request formats, response formats, validation rules, and error handling.

Browser UI Requirements

Create a simple browser UI connected to the Spring Boot backend. This should be generated using Kiro/Amazon Q

The UI must allow users to:

- View a dashboard
- Create a festival area
- Submit a crowd report
- View recent reports
- View and resolve active alerts.

Storage Requirement

For the minimum version, data may be stored in memory. Data does not need to survive an application restart.

Business Rules

The application must enforce these rules:

- A crowd report must belong to an existing area.
- Invalid input should be rejected.
- A 'FULL' crowd report should create an active alert.
- Resolved alerts should not appear as active.
- Avoid creating duplicate active alerts for the same area.

Testing Requirements

Provide tests for the most important behaviour, such as:

- Creating and listing areas
- Submitting a valid crowd report
- Rejecting a report for a missing area
- Creating an alert from a 'FULL' report
- Resolving an alert

Minimum Expected Deliverable

By the end of the day, each group should aim for:

- A running Spring Boot application
- REST APIs for the main workflows
- A browser UI connected to the backend
- In-memory storage
- Basic validation and error handling
- A small set of passing tests

Optional Stretch Goals

Only attempt these after the minimum application is working.

Stretch Goal 1: Database Persistence

Replace in-memory storage with a relational database.

Stretch Goal 2: Kafka Alert Flow

When a crowd report is submitted, publish an event to Kafka. A Kafka consumer should create the alert when the crowd level is `FULL`.